A Guided Companion to "Ancilla-mediated fixed-point quantum search using Grover iterations"

Every concept, variable and equation explained — with a full $N = 8,\ M = 1$ worked example,
what changed from the first version, and why the corrected paper is consistent

Who this is for. You know what a qubit, a superposition, and a measurement are, and you have heard of Grover's algorithm. Everything else is built up from there. Read Part 1 for the story, Part 2 for the meaning of every symbol and equation, Part 3 for a complete numerical run, Part 4 for why an unknown θ is not a problem, and Parts 5–6 for what was wrong in the first version and how it was fixed.

1 The big picture, in plain words

The task. You have a database of N items. Exactly M of them are "solutions" (the things you want). You cannot look inside; you only have an *oracle* — a black box that, shown an item, tells you (quantum-mechanically) whether it is a solution. Classically you must try about N/M items. Grover's quantum algorithm finds one in about $\sqrt{N/M}$ oracle uses — a quadratic speed-up.

How Grover works, geometrically. Put the register in an equal superposition of all N items. Think of the whole quantum state as an *arrow* living in a 2-D plane whose axes are "non-solutions" ($|R\rangle$) and "solutions" ($|S\rangle$). The arrow starts almost flat (mostly non-solution), tilted up by a tiny angle θ from the $|R\rangle$ axis. Each Grover step rotates the arrow by a fixed angle 2θ towards $|S\rangle$. When the arrow points along $|S\rangle$ (angle $\pi/2$), a measurement returns a solution with near-certainty.

The catch — the "soufflé problem". You must stop at exactly the right number of steps. Rotate too far and the arrow swings *past* $|S\rangle$ and back down — the success probability rises, peaks, then falls, like a soufflé that collapses if you leave it in the oven too long. The right number of steps depends on M, so **if you don't know M, you don't know when to stop.**

This paper's idea. Add one extra qubit, an *ancilla*, used as a "success flag." A specially built operator $\hat{F}(\theta)$ copies the current solution-amplitude onto the ancilla and then *measures only the ancilla*. Two things can happen:

- Flag reads $|1\rangle$: the data register has collapsed onto the solution — you are done.
- Flag reads $|0\rangle$: you have *not* destroyed the data. Instead the data register is gently "rewound" to a known, fixed angle 2θ, from which you can safely try again.

Because a failed check rewinds to a known state instead of ruining it, you can keep trying with a *doubling schedule* of amplification lengths $(0, 0, 1, 3, 7, \ldots$ Grover steps) until the flag fires. This gives a guaranteed success probability ($\geq 92.6\%$) *without knowing* M, while keeping the $\sqrt{N/M}$ speed-up. That is the whole paper.

2 Every concept, variable and equation

2.1 The search space and the oracle

- N — number of items in the database (paper uses $N = 10^{10}$ for plots). Usually $N = 2^n$ for n qubits.
- M — number of solutions (marked items). *Unknown* in the setting of interest.
- $|x\rangle$ — a computational-basis state, one specific item.
- $\hat{O}$ — the **oracle**. *Significance:* it is the only place the problem enters. Eq. (1) of the paper,

$$\hat{O}|x\rangle = \begin{cases} -|x\rangle, & x \text{ a solution} \\ +|x\rangle, & \text{otherwise} \end{cases} \qquad \Longleftrightarrow \qquad \hat{O} = \hat{I} - 2\sum_{s\in\mathcal{S}} |s\rangle\langle s|,$$

  says the oracle flips the *sign* (phase) of solution states and leaves the rest alone. $\mathcal{S}$ is the set of solutions. Flipping a sign costs nothing classically visible, but in superposition it is what lets interference single out the solutions. "One oracle call" is the paper's unit of cost.

2.2 Collective states, the angle θ, and state preparation

- $|S\rangle$ — the normalised superposition of *all* solution items (Eq. 7); $|R\rangle$ — the normalised superposition of all non-solutions (Eq. 8). Together they span the 2-D plane the "arrow" lives in. $|S|, |R|$ are just the normalising constants; s_i, r_i are the individual amplitudes (for uniform weights, $s_i = 1$ and $|S| = \sqrt{M}$).
- θ — the **Grover angle**, Eq. (5): $\theta = \arcsin\sqrt{M/N}$. *Significance:* $\sin^2\theta = M/N$ is exactly the probability of measuring a solution from the initial equal superposition. A big M means a big θ (easy search); a lone needle ($M = 1$) means a tiny θ (hard search).
- $\hat{P}$ — the **state-preparation** operator; $\hat{P}|0\rangle^{\otimes n} = |\psi\rangle = \cos\theta\,|R\rangle + \sin\theta\,|S\rangle$ (Eq. 2). For an equal superposition $\hat{P} = H^{\otimes n}$ (Hadamards). *Significance:* it defines where the arrow starts and, crucially, its *inverse* $\hat{P}^\dagger$ lets us build the "reflection about $|\psi\rangle$" used everywhere below.
- $|\psi\rangle$ — the initial data state (the starting arrow, angle θ).

2.3 The Grover / amplitude-amplification operator $\hat{G}$

$\hat{G}$ is one Grover iteration. Its whole effect (Eq. 3–4) is a rotation:

$$\hat{G}^k|\psi\rangle = \cos\big((2k{+}1)\theta\big)|R\rangle + \sin\big((2k{+}1)\theta\big)|S\rangle.$$

- k — how many Grover steps you apply. Each step adds 2θ to the arrow's angle.
- ϵ — the **optimal** number of steps if M *were* known (Eq. 6): $\epsilon = \left\lfloor \frac{\pi}{4}\sqrt{N/M} - \frac{1}{2} \right\rfloor$. It is the k that lands $(2k{+}1)\theta$ closest to $\pi/2$ (arrow on $|S\rangle$). This is the benchmark "best you could possibly do."

Fig. 2 (the geometry) in one line. The arrow starts at θ; each $\hat{G}$ turns it by 2θ; the target is $\pi/2$; turning past $\pi/2$ "overshoots" and lowers the success amplitude (that is the soufflé problem); and a failed fixed-point check (below) snaps the arrow back to 2θ.

2.4 The ancilla and the fixed-point operator $\hat{F}(\theta)$

Now add one ancilla qubit a. States are written $|\text{ancilla}\rangle|\text{data}\rangle$, e.g. $|0_a\rangle|S\rangle$. The starting state is $|\Psi\rangle = |0_a\rangle(\cos\theta|R\rangle + \sin\theta|S\rangle)$ (Eq. 9).

- $\mathcal{V}$ — the **invariant subspace** (Eq. 12), spanned by the four states $|0_a\rangle|R\rangle$, $|0_a\rangle|S\rangle$, $|1_a\rangle|R\rangle$, $|1_a\rangle|S\rangle$. *Significance:* although the full space has $2N$ dimensions, the algorithm never leaves these four directions, so *all the physics is* 4×4. This is the single most important simplification.
- $\hat{F}(\theta)$ — the **fixed-point operator**, given as the 4×4 matrix in Eq. (13). What it *does* is Eq. (14):

  $$\hat{F}(\theta)\,|0_a\rangle|R\rangle = -\cos 2\theta\,|0_a\rangle|R\rangle - \sin 2\theta\,|0_a\rangle|S\rangle, \qquad \hat{F}(\theta)\,|0_a\rangle|S\rangle = |1_a\rangle|S\rangle.$$

  Read it as two behaviours: on the non-solution part it *rotates the data* (by 2θ, staying on the ancilla-0 side); on the solution part it *flips the ancilla to* $|1\rangle$. Acting on the full start state (Eq. 16):

  $$\hat{F}(\theta)|\Psi\rangle = \underbrace{-\cos\theta\,|0_a\rangle\big(\cos 2\theta\,|R\rangle + \sin 2\theta\,|S\rangle\big)}_{\text{flag }=0\text{ branch}} + \underbrace{\sin\theta\,|1_a\rangle|S\rangle}_{\text{flag }=1\text{ branch}}.$$

The magic of the reset. Measure the ancilla. *(i)* With probability $\sin^2\theta$ you get $|1_a\rangle$ — the data is now exactly $|S\rangle$: **success**. *(ii)* With probability $\cos^2\theta$ you get $|0_a\rangle$ and the data collapses to $\cos 2\theta\,|R\rangle + \sin 2\theta\,|S\rangle$ (Eq. 17), i.e. **the arrow at the fixed angle** 2θ — *no matter what angle it had before.* A failed check does not waste your work; it rewinds to a known checkpoint. This is why the method is called "fixed point."

The symbols $|\Psi'\rangle$ (Eq. 17), $|\Phi\rangle, |\phi\rangle$ (angle ϕ, a general mid-run state, Eq. 18), and $|\Phi_m\rangle, |\phi_m\rangle$ (Eq. 20) are all just names for "the data state after a measurement"; the key fact is that a failed check always returns the angle-2θ state.

2.5 How $\hat{F}$ is actually built (gate level)

Eq. (15) factors $\hat{F} = \hat{D}\,\hat{C}_S$ (apply $\hat{C}_S$ first):

- $\hat{C}_S = H_a\,(\text{c-}\hat{O})\,H_a$ — the **flag step**. A Hadamard puts the ancilla in $|+\rangle$; an oracle *controlled by the ancilla* stamps a -1 only on the (ancilla, solution) branch; a second Hadamard turns that phase into a bit-flip. Net effect: the ancilla flips to $|1\rangle$ exactly when the data is a solution. (H_a = Hadamard on ancilla; c-$\hat{O}$ = ancilla-controlled oracle.)
- $\hat{D} = \hat{P}\,\hat{R}_0^{(a=0)}\,\hat{P}^\dagger$ — the **reflection step**, active only when the ancilla is $|0\rangle$. $\hat{R}_0^{(a=0)}$ flips the sign of $|0_a\rangle|0\rangle^{\otimes n}$; sandwiching it by $\hat{P}^\dagger \ldots \hat{P}$ turns it into the reflection about $|\psi\rangle$ that produces the 2θ rotation.

Significance: this shows $\hat{F}$ needs only the oracle, $\hat{P}, \hat{P}^\dagger$, and elementary gates — and **one (controlled) oracle call** — so it never needs the value of θ or M.

2.6 The doubling schedule and Algorithm 1

Because a failed check rewinds to 2θ, we can amplify a controlled amount before each check.

- ϵ_0 (Eq. 22) — the worst-case optimum, at $M = 1$: the largest number of Grover steps you could ever need.
- a (Eq. 24) — how many checks the schedule needs, $a = \lceil \log_2(\pi\sqrt{N}) \rceil$, so the doubling always reaches past the peak.

- $\varphi_j = 2^j\theta$ (Eq. 23) — the **angle at the j-th check**, $j = 0, 1, \ldots, a-1$. The angles *double*: $\theta, 2\theta, 4\theta, 8\theta, \ldots$
- g_j (Eq. 25) — the number of Grover steps applied *before* check j, starting from the reset angle 2θ: $g_0 = 0$, $g_j = 2^{j-1} - 1$. (Why: from 2θ you need g_j steps to reach $2^j\theta$.)
- $\mathcal{S}_A$ (Eq. 26) — the list of solution amplitudes reached, $\{\sin\theta, \sin 2\theta, \sin 4\theta, \ldots\}$.

Algorithm 1 is then: prepare $|\Psi\rangle$; for $j = 0, \ldots, a-1$: apply g_j Grover steps, apply $\hat{F}$, measure the ancilla; if $|1\rangle$ stop (found), else the state has reset to 2θ and you continue. If all checks fail, no solution exists.

2.7 Why it works: the 92.6% guarantee

- $p_s = \sin^2(2^j\theta)$ (Eq. 27) — probability the j-th check succeeds (given you reached it). Failure probability is $\cos^2(2^j\theta)$.
- c (Eq. 28) — because the angles double, exactly one check lands in the "good window" $[\frac{\pi}{4}, \frac{\pi}{2})$; call its index c, so $\frac{\pi}{4} \le 2^c\theta < \frac{\pi}{2}$.
- $p_{c,1}$ (Eqs. 29–30) — the probability of succeeding within the two consecutive good checks $c, c+1$. Writing $x = 2^c\theta$,

$$p_{c,1}(x) = \sin^2 x + \sin^2(2x)\cos^2 x = 1 - \cos^2 x \, \cos^2(2x).$$

  Minimising the right-hand form over $x \in [\frac{\pi}{4}, \frac{\pi}{2})$ gives the **worst case**: minimum value 0.92593 at $x \approx 1.150$ (this is Fig. 6). *Significance:* this single number is the paper's headline guarantee — no matter what M is, two well-placed checks already succeed with probability ≥ 0.9259.
- P_{S_j} (Eq. 31) and P_j (Eq. 32) — probability of first success at check j, and cumulative success by check j. The cumulative one telescopes to the clean closed form $P_j = 1 - \prod_{m=0}^{j} \cos^2(2^m\theta)$, from which $P_j \ge p_{c,1} \ge 0.9259$ (Eq. 33). Fig. 7 (vs. M) and Fig. 8 (vs. checkpoint) show these probabilities.

2.8 Cost: the query complexity

- $o_j = 1 + g_j$ (Eq. 34) — oracle calls used *at* check j (one for the $\hat{F}$ check plus g_j Grover steps): $o_0 = 1$, $o_j = 2^{j-1}$.
- $O_j = \sum_{l \le j} o_l = 2^j$ (Eq. 35) — *cumulative* oracle calls to reach check j. This tidy "2^j" is the corrected bookkeeping (see Part 6).
- $O_T = 2^{c+1} < \pi\sqrt{N/M}$ (Eq. 36) — worst-case total to hit the guarantee. Hence **worst-case complexity** $\mathcal{O}(\pi\sqrt{N/M})$.
- $\mathbb{E}(O_j)$ (Eqs. 37–39) — the *expected* cost, which works out to $\approx 1.33\,\epsilon \sim \sqrt{N/M}$: only about a third more than if you had known M all along. Fig. 9 shows this.
- If $M = 0$ (no solution): all checks fail and you stop after $\mathcal{O}(\sqrt{N})$ calls — the algorithm certifies non-existence.

Table I compares this with Grover's $\pi/3$ method, Boyer et al. (trial-and-error), and Yoder/Li (phase-matching). The paper's row: deterministic, real-plane, $\mathcal{O}(\pi/\sqrt{\lambda_0})$ worst case with $\lambda_0 = M/N$, success $\ge 92.6\%$.

3 A complete worked example: $N = 8,\ M = 1$

Take $n = 3$ qubits, $N = 8$ items, one marked item ($M = 1$), say $|S\rangle = |101\rangle$. Let the non-solution collective state $|R\rangle$ be the equal superposition of the other seven items. Then $\sin^2\theta = 1/8$, so

$$\theta = \arcsin\sqrt{1/8} = 0.3614 \text{ rad} = 20.7^\circ, \qquad \cos 2\theta = 0.75,\ \sin 2\theta = 0.6614.$$

The schedule needs $a = \lceil \log_2(\pi\sqrt{8}) \rceil = 4$ checks, at angles $\theta, 2\theta, 4\theta, 8\theta$.

Start. $|\Psi\rangle = |0_a\rangle\big(\cos\theta\,|R\rangle + \sin\theta\,|S\rangle\big)$ with $\cos\theta = 0.9354,\ \sin\theta = 0.3536$. The arrow is at 20.7°; the chance a naive measurement already hits the solution is $\sin^2\theta = 1/8 = 12.5\%$.

Round $j = 0$ (angle θ, no amplification, $g_0 = 0$). Apply $\hat{F}$:

$$\hat{F}|\Psi\rangle = -\underbrace{0.9354}_{\cos\theta}|0_a\rangle\big(0.75\,|R\rangle + 0.6614\,|S\rangle\big) + \underbrace{0.3536}_{\sin\theta}|1_a\rangle|S\rangle.$$

Measure the ancilla. $P(|1_a\rangle) = \sin^2\theta = 0.125 \to$ *done*. Otherwise (0.875) the data resets to the arrow at $2\theta = 41.4^\circ$.

Round $j = 1$ (angle 2θ, still $g_1 = 0$). The reset already put us at 2θ, so we check immediately. Now the incoming angle is $\phi = 2\theta$, and $\hat{F}$ gives $|1_a\rangle$-amplitude $\sin 2\theta = 0.6614$, so this check succeeds with probability $\sin^2 2\theta = 0.4375$. On failure, reset to 2θ again.

Round $j = 2$ (angle 4θ, $g_2 = 1$). From the reset 2θ, apply *one* Grover step ($+2\theta$) to reach $4\theta = 82.8^\circ$ — almost on $|S\rangle$! Apply $\hat{F}$: success probability $\sin^2 4\theta = 0.9844$. This is essentially certain convergence.

Round $j = 3$ (angle 8θ, $g_3 = 3$). Rarely reached; 8θ has overshot to 165.6°, illustrating why we did not need to go this far.

Table 1: *
State evolution for $N = 8, M = 1$ (corrected schedule). "cum. P" is the running probability of having succeeded by that round; "O_j" is the total oracle calls spent to reach it.

check j	Grover g_j	angle $2^j\theta$	(deg)	$p_s = \sin^2$	cum. P_j	O_j
0	0	0.3614	20.7°	0.1250	0.1250	1
1	0	0.7227	41.4°	0.4375	0.5078	2
2	1	1.4455	82.8°	0.9844	**0.9923**	4
3	3	2.8909	165.6°	0.0615	0.9928	8

Read the table like this. After at most three checks — costing only $O_2 = 4$ oracle calls — the cumulative success probability is 99.2%, far above the guaranteed floor 92.6%, *and we never needed to know $M = 1$*. The "good window" index here is $c = 2$ (angle $4\theta = 82.8^\circ$ lies in $[45^\circ, 90^\circ)$), exactly where the table shows the big jump. For comparison, if you *did* know M, Grover's optimum is about $k^* \approx 1.7$ steps; $k = 2$ gives 94.5%, and over-running to $k = 3$ collapses it to 33% — the soufflé problem the fixed-point schedule sidesteps.

4 "If M is unknown, so is θ" — why that is not an anomaly

A natural worry: the operator $\hat{F}(\theta)$ and the preparation $\hat{P}$ are written *as functions of* θ, yet $\theta = \arcsin\sqrt{M/N}$ depends on the unknown M. If you cannot compute θ, how can you build the circuit?

Resolution. You never build anything from θ. The circuit is made of θ-*free* gates — Hadamards, the (controlled) oracle, and a reflection about $|0\rangle^{\otimes n}$ — and the angle θ *emerges* from running them. Writing $\hat{F}(\theta)$ is a *description* of what the fixed gates do to the state, not a *recipe* that consumes θ as an input.

The gate recipe is

$$\hat{F} = \underbrace{\left[\hat{P}\,\hat{R}_0^{(a=0)}\,\hat{P}^\dagger\right]}_{\hat{D}}\;\underbrace{\left[H_a\,(\text{c-}\hat{O})\,H_a\right]}_{\hat{C}_S}, \qquad \hat{P} = H^{\otimes n}.$$

Look at every ingredient: $\hat{P} = H^{\otimes n}$ is just Hadamards (*no* θ, *no* M); $\hat{R}_0$ flips the sign of $|0\rangle^{\otimes n}$ (fixed); H_a is a Hadamard; and c-$\hat{O}$ is the oracle you were *given* as a black box — you never wrote θ into it. The number M lives only inside the sealed oracle. The angle θ is then an *emergent overlap*: $|R\rangle$ and $|S\rangle$ are defined by whatever the oracle marks, and $\langle\psi|S\rangle = \sqrt{M/N} = \sin\theta$ is fixed automatically by that geometry.

Demonstration: the same gates, three different problems. To make this concrete I built the *identical* matrix $\hat{F}$ above for $N = 8$ and applied it to $|0_a\rangle|\psi\rangle$ for three different (secret) marked sets. Nothing θ-valued was ever entered; only the oracle's marked set differs.

Table 2: *
One fixed circuit, three problems ($N = 8$). Every row uses the *same* gates $\hat{F} = [\hat{P}\hat{R}_0\hat{P}^\dagger][H_a(\text{c-}\hat{O})H_a]$ with $\hat{P} = H^{\otimes 3}$; only the oracle changes. The angle θ and everything to its right are *outputs*, read off the resulting state — never inputs.

marked set (secret)	M	$\theta = \arcsin\sqrt{M/N}$	2θ	$\cos 2\theta$	$\sin 2\theta$	$P(\text{flag}{=}1)$	checks a
$\{5\}$	1	0.3614 (20.7°)	41.4°	0.750	0.661	0.125	4
$\{5, 6\}$	2	0.5236 (30.0°)	60.0°	0.500	0.866	0.250	4
$\{4, 5, 6, 7\}$	4	0.7854 (45.0°)	90.0°	0.000	1.000	0.500	4

In each case the output state came out numerically equal to

$$\hat{F}\,|0_a\rangle|\psi\rangle = -\cos\theta\,|0_a\rangle\big(\cos 2\theta\,|R\rangle + \sin 2\theta\,|S\rangle\big) + \sin\theta\,|1_a\rangle|S\rangle,$$

with the θ appearing there being exactly $\arcsin\sqrt{M/N}$ — a value that was *never supplied*. The columns $\cos 2\theta, \sin 2\theta, P(\text{flag}{=}1)$ come out different for $M = 1, 2, 4$ *from the same matrix*, precisely as the geometry predicts.

Two things this makes concrete. (i) The schedule length $a = \lceil \log_2(\pi\sqrt{N})\rceil = 4$ is the *same* in all three rows — it depends only on N, never on M — so you fix the number of checkpoints from N alone, and the algorithm still adapts to whatever M turns out to be, because each checkpoint measures the flag angle $2^j\theta$ the circuit *produces* rather than one you computed in advance. (ii) The $M = 4$ row is the clean limiting case: $2\theta = 90°$, so a single application rotates fully onto $|S\rangle$ ($\sin 2\theta = 1$) and the flag fires with probability $\frac{1}{2}$ — the search is essentially done in one step, again with no knowledge of M used anywhere.

So θ being unknown is harmless *precisely because nothing in the circuit needs to know it.* The fixed-point construction reads θ off the state at each checkpoint (through the flag-measurement statistics) instead of requiring it as input — which is exactly what lets a *single*, problem-agnostic circuit solve every instance.

5 What changed from the first version, and why it is now consistent

The first version (`mainwithoutappendix.tex`) had the right *idea* but several mutually inconsistent pieces: the written operator, the schedule, and the proof did not describe the same algorithm, so the advertised "92.6% in $\mathcal{O}(\pi\sqrt{N/M})$" was not actually delivered by the stated procedure. Here is each problem and its fix.

1. The fixed-point operator was written inconsistently (and non-unitary). The old Eq. (12) gave $\hat{F}$ as a $2N \times 2N$ matrix built from identity blocks that coupled $|0_a\rangle|R\rangle \leftrightarrow |1_a\rangle|R\rangle$ and swapped $|0_a\rangle|S\rangle \leftrightarrow |1_a\rangle|S\rangle$. Applied to $|\Psi\rangle$ this produces a spurious $|1_a\rangle|R\rangle$ term and *no* $|0_a\rangle|S\rangle$ term — i.e. it contradicts the operator's own stated action, Eq. (13). Worse, read literally it sends every non-solution basis state to the *same* collective $|S\rangle$, so different columns are not orthogonal (their overlap is $\sin^2 2\theta \neq 0$): the matrix is *not unitary*, so it cannot be a valid quantum operation.

Fix. Present $\hat{F}$ as the explicit 4×4 unitary on the invariant subspace $\mathcal{V}$ (new Eq. 13), verify $\hat{F}^\dagger\hat{F} = I_4$, and add a gate-level derivation (Eq. 15) proving the circuit really implements the intended action. *Consistency restored:* the operator, its matrix, and its action now agree, and it is manifestly unitary.

2. The schedule did not achieve the peak (the central bug). The old set was $\mathcal{K} = \{2^j - 2\} = \{0, 2, 6, 14, \dots\}$ *and* the algorithm applied $\hat{F}$ *before* amplifying. Combined with the reset-to-2θ, the actual angles checked were $\{\theta, 2\theta, 6\theta, 14\theta, \dots\}$ — which do *not* double cleanly and can jump right over the good window $[\frac{\pi}{4}, \frac{\pi}{2})$. The paper's proof, however, assumed clean doubling $\{2^c\theta\}$. So the proof described a different schedule than the algorithm ran. Also, applying $\hat{F}$ before the last (largest) amplification meant the biggest boost was never measured. For $N = 8$ the old angles are $\theta, 2\theta, \mathbf{6}\theta(124^\circ), 14\theta$ — the third check has *overshot* to 124° (success only 0.68), whereas the corrected $4\theta = 82.8^\circ$ gives 0.98.

Fix. Amplify *then* check, with $g_0 = 0$, $g_j = 2^{j-1} - 1$, so the checked angles are exactly $2^j\theta$ (clean doubling). Now the algorithm performs precisely the angles the proof assumes, and one of them always lands in the good window. *This is what makes the* 92.6% *guarantee true of the algorithm as written.* (Verified numerically: worst-case success = 0.9259.)

3. Smaller mathematical corrections.

- *Wrong peak condition.* The text said the amplitude peaks at $(\theta + 2k\theta) = \pi/4$; it peaks at $\pi/2$. (The formula for ϵ was right; only the stated condition was wrong.)
- *Off-by-one in the cost sum.* The old $O_j = 2 \cdot 2^j - j$ did not match its own summation; the corrected cumulative cost is the clean $O_j = 2^j$.
- *Undefined "no-solution" cost.* The old text wrote the $M = 0$ cost as $\mathcal{O}(\pi\sqrt{N/M})$, which is infinite/undefined at $M = 0$; corrected to $\mathcal{O}(\sqrt{N})$.
- *Rigorous bound.* The worst-case argument now uses the identity $p_{c,1} = 1 - \cos^2 x \cos^2 2x$ and the telescoped $P_j = 1 - \prod \cos^2$, giving a clean proof that $P_j \geq 0.9259$.
- *Fair comparison and related work.* The comparison now notes that Yoder/Li deliver a *tunable* $1 - \delta^2$ (their constants are quoted at fixed δ), and a paragraph places the method against earlier measurement-based ancilla schemes (Grover–Patel–Tulsi; Mani–Patvardhan).

Why the corrected paper is internally consistent. Three things now describe *the same* algorithm: (i) the operator $\hat{F}$ (unitary, with a gate-level proof of its action); (ii) the schedule (amplify-then-check at angles $2^j\theta$); and (iii) the analysis (which assumes exactly those angles). Every downstream number — the 0.9259 floor, the $O_j = 2^j$ bookkeeping, the worst-case $\pi\sqrt{N/M}$, and the expected $\sqrt{N/M}$ — follows from that one consistent schedule, and each was checked numerically.

6 The Python code: what was wrong and what fixed it

The notebook that generates the cost figure computes the expected number of oracle calls, $\mathbb{E}(O_j) = \sum_j O_j \times \Pr(\text{first success at } j)$. The original `calculate_exp` got *both* factors wrong.

Bug A — wrong per-round cost. It used

```
oc = 1
for q in range(l):
    oc += (2 ** q - 1) # = 2**l - l (old, wasteful schedule)
```

which is the cost of the old $\{2^j - 2\}$ schedule, not the corrected one. The right cumulative cost is simply $O_l = 2^l$.

Bug B — wrong probability weight. It accumulated the cost against the *cumulative* success probability `p` instead of the probability of *first* success in that round:

```
p += (sin(2**l*theta)**2) * fp # p is the running cumulative success
exp_O_j += oc * p # <-- multiplies cost by cumulative p (wrong)
```

An expectation must weight each round's cost by $\sin^2(2^l\theta)\prod_{m<l}\cos^2(2^m\theta)$ (the chance you *first* succeed exactly then), not by the running total. The two bugs together inflated and distorted the curve (the reported overhead swung between 1.6× and 3.4× the optimum depending on M).

The fix. Use $O_l = 2^l$ and the first-success weight:

```
def calculate_exp(theta):
    l, fp, E, lastO = -1, 1.0, 0.0, 1
    while theta * 2**l < pi/2:
        l += 1
        O_l = 2**l # corrected cost
        E += O_l * (sin(2**l*theta)**2) * fp # cost x P(first success at l)
        fp *= cos(2**l*theta)**2 # carry failure probability
        lastO = O_l
    E += fp * lastO # never-converged tail
    return E
```

With this, $\mathbb{E}(O_j)/\epsilon \approx 1.33$ across all M (mean 1.315), i.e. $\mathbb{E}(O_j) \sim \sqrt{N/M}$ — matching the corrected text and Fig. 9. (The probability routine `calculate_CP`, which computes $P = 1 - \prod\cos^2$, was already correct and was left unchanged; it produces Figs. 7–8.)

One-sentence summary. The first version had a good idea wired up with three parts that quietly disagreed — the operator, the step schedule, and the proof; the corrected version makes all three describe the identical amplify-then-check, angle-doubling algorithm, fixes the cost arithmetic in the paper and the notebook, and every headline number then follows and checks out.